

Auron: Depth-Efficient Language Models via Hybrid Recurrent-Attention Weight Sharing

The Chimera Topology

Florian Gasquez*

Cube Digital Media Ltd / Soulkyn

<https://fyx.jp> | <https://soulkyn.com>

Code: <https://github.com/Fy-/Auron> | Models: <https://huggingface.co/nyxia>

March 2026

Draft — multi-scale training ongoing

Abstract

We present the **Chimera Topology**, a hybrid language model architecture that decouples virtual depth from physical parameter count through selective weight sharing. The architecture combines Gated Delta Networks (GDN) for $O(n)$ recurrent processing with standard Grouped-Query Attention (GQA) in a 3:1 ratio, organized into a two-zone stack: unique bottom layers for token parsing and shared top layers for iterative reasoning. We train three model scales—279M, 510M, and 1.1B total parameters—on a mixed pretraining dataset of 5B tokens across four sources. At 510M parameters, the model achieves a validation loss of 3.16 at step 103.5K (~ 3.39 B tokens), significantly outperforming the 279M model at 58.5K steps (3.41, ~ 1.92 B tokens), confirming that the Chimera topology scales efficiently with width.

Key findings include: (1) a critical embedding learning rate bottleneck specific to large-vocabulary small models, (2) expected WSD stable-phase loss plateaus resolved by the warmdown phase, (3) emergent ChatML stop-token behavior from just 2% dialogue pretraining data, and (4) hardware scaling characteristics across DGX Spark, H100, and B200 GPUs showing the architecture’s compute-bound nature at small scale transitioning to memory-bound at 1B+. We also propose **Chimera-MoE**, a future extension combining sparse Mixture-of-Experts routing in the bottom layers with the dense shared top, achieving extreme virtual-to-active parameter ratios on consumer hardware.

1 Introduction

The dominant approach to improving language model capability is to increase parameter count, requiring proportionally more compute, memory, and training data [Hoffmann et al., 2022]. For deployment on edge hardware or resource-constrained environments, this scaling path is impractical. Recent work on sub-billion parameter models [Liu et al., 2024, Ben Allal et al., 2025] has focused primarily on data curation and training recipes, largely preserving the standard dense transformer architecture.

We take a different approach: rather than optimizing data for a fixed architecture, we redesign the architecture to exploit the mathematical properties of different layer types and the inherent redundancy of deep transformer representations. The Chimera Topology achieves its efficiency through three mechanisms:

*Corresponding author. florian@soulkyn.com

1. **Selective weight sharing**—different layer types (recurrent vs. attention) are shared at independent frequencies based on their tolerance for recursion.
2. **Two-zone architecture**—unique early layers handle dense token parsing while shared later layers perform iterative reasoning.
3. **Large vocabulary offloading**—a 152K-token vocabulary (Qwen 3) handles morphology and spelling entirely in the embedding lookup, freeing the reasoning core to focus on semantics.

This paper describes the architecture, its theoretical motivation, experimental results from three concurrent pretraining runs at different scales, and practical discoveries about optimization dynamics that may generalize to other weight-shared models.

2 Related Work

2.1 Weight Sharing in Transformers

The Universal Transformer [Dehghani et al., 2018] first demonstrated that recursively applying the same transformer block across depth could match or exceed standard transformers on algorithmic tasks. ALBERT [Lan et al., 2019] scaled this to BERT-class pretraining, showing that full cross-layer parameter sharing with factorized embeddings achieved competitive downstream performance with $18\times$ fewer parameters.

Saunshi et al. [2025] prove that a k -layer transformer looped L times nearly matches a kL -layer non-looped model on synthetic reasoning tasks. Most directly relevant to our work, Zhu et al. [2025] introduce the Ouro architecture, establishing recursion count as a third scaling axis alongside parameters and data. Their Ouro-1.4B model with 4 loops matches 4B-class models, with gains concentrated in reasoning tasks rather than factual recall.

Bae et al. [2024] demonstrate that pretrained models can be converted to recursive form using depth-wise LoRA adaptors, achieving $2\text{--}3\times$ inference throughput—suggesting that weight sharing reflects genuine redundancy in deep transformer representations.

2.2 Hybrid Recurrent-Attention Architectures

Griffin [De et al., 2024] combines a Real-Gated Linear Recurrent Unit with local attention, matching Llama-2 quality trained on $6\times$ fewer tokens. Jamba [Lieber et al., 2024] interleaves Mamba blocks with attention at a 7:1 ratio, fitting 256K context in a single 80GB GPU.

Zamba [Glorioso et al., 2024a] is particularly relevant: it uses a Mamba backbone with a single attention layer weight-shared across the full network depth, establishing that attention sharing is viable when recurrent layers provide primary processing. Zamba2 [Glorioso et al., 2024b] extends this with two shared attention blocks in an alternating pattern, adding LoRA projectors per position for differentiation.

Our architecture follows this lineage but inverts the ratio—we use GDN (a stronger recurrent primitive than Mamba) as the primary layer type and standard GQA attention as the sparse global mechanism, shared independently at different frequencies.

2.3 Gated Delta Networks

Gated Delta Networks [Yang et al., 2024] combine data-dependent gating (adaptive memory erasure) with the delta update rule (precise key-value association). GDN outperforms Mamba2 across standard benchmarks including in-context retrieval and is used in production models (Qwen 3.5). GDN is implemented in the flash-linear-attention library [Yang et al., 2023], which provides hardware-efficient chunked training kernels.

The key property making GDN suitable for weight sharing is its recurrent state: unlike attention, which produces independent outputs per layer, GDN’s hidden state naturally evolves across iterations. When the same GDN block processes the hidden state multiple times, each pass operates on a legitimately different input—the accumulated recurrent state from prior loops. Attention lacks this property; its outputs depend only on the current input, making repeated application of the same weights degenerative.

2.4 Optimization

We use the Muon optimizer [Jordan et al., 2024], which applies Newton-Schulz orthogonalization to gradient matrices, approximating the spectral-norm steepest descent direction. Bernstein & Newhouse [2024] reframe Adam, Shampoo, and Prodigy as steepest descent under different operator norms. Liu et al. [2025] demonstrate that Muon scales to large models (3B–16B) with $\sim 2\times$ compute efficiency over AdamW.

For memory efficiency, we employ fused linear cross-entropy [Hsu et al., 2024] via the Liger-Kernel library, avoiding materialization of the full $[\text{batch} \times \text{seq_len} \times \text{vocab}]$ logit tensor. With a 152K vocabulary, this tensor would consume 9.4 GB per forward pass at batch size 16.

3 Architecture

3.1 Overview

The Chimera Topology organizes the network into two functional zones within a standard pre-norm transformer framework:

- **Bottom zone (“Retina”)**: N unique HybridBlocks for dense token parsing—analogue to retinal preprocessing before visual cortex.
- **Top zone (“Brain”)**: P physical HybridBlocks looped L times for iterative reasoning—weight sharing as learned recurrence.

Total virtual depth = $N + P \times L$ layers. Each HybridBlock follows a 3:1 GDN:Attention pattern controlled by `attn_interval=4`: blocks at positions 0, 1, 2 use GDN; block at position 3 uses GQA attention.

3.2 Model Configurations

Table 1: Model configurations for the three Auron scales.

Parameter	Auron-279M	Auron-510M	Auron-1.1B
Dimension	1024	1536	2048
Bottom layers (unique)	4	4	6
Top layers (physical)	4	4	6
Top loops	3	3	3
Virtual depth	16	16	24
Attention heads (GQA)	16 (4 KV)	24 (6 KV)	32 (8 KV)
GDN heads (head dim)	16 (64)	24 (64)	32 (64)
GDN V expansion	2	2	2
FFN mult (SwiGLU hidden)	2.67 (2734)	2.67 (4101)	2.67 (5472)
RoPE partial factor	0.25	0.25	0.25
Vocabulary	151,936	151,936	151,936
Sequence length	2048	2048	2048
Unique params	123M	277M	761M
Embedding params	155M	233M	311M
Total params	279M	510M	1.1B
Virtual equivalent	~350M	~787M	~1.8B

All three configurations share the same topology pattern ($4+4\times 3$ or $6+6\times 3$), differing only in width. This enables clean scaling analysis.

3.3 Why Two Zones?

Early experiments with full Ouroboros (all layers shared) revealed a parsing bottleneck: the model struggled to map dense multi-byte tokens from the 152K vocabulary into stable semantic representations using recursive weights. The shared layers exhibited “geometric gravity”—a tendency to pull hidden states toward the most statistically dominant local patterns, producing tautological repetition.

Dedicating the first N layers as unique, unshared blocks solves this by providing a clean, non-recursive parsing stage. Once token features are extracted into stable representations, the shared top section can focus entirely on reasoning through iteration.

3.4 Convergence Accelerators

Weight sharing introduces three failure modes that require structural countermeasures:

Layer ID Gate Injection. Each virtual layer in the top zone receives a unique learnable embedding $\mathbf{e}_{\text{layer}}[l, b]$ of shape $(\text{dim},)$. This embedding is injected into the GDN mixer input only—not the residual stream. This modulates the GDN gate without accumulating variance in the representations read by attention layers.

$\mathbf{x}_0$ Injection. The output of the initial embedding layer ($\mathbf{x}_0$) is multiplied by a per-layer learnable scalar λ_l and added to the input of every virtual layer. This ensures the original token context remains accessible regardless of how many loops the hidden state has traversed.

U-Net Skip Connections. Learnable skip connections bridge encoder-half activations to the corresponding decoder-half positions (layer 0 to layer 15, etc.), providing direct gradient highways for the deep virtual stack.

3.5 Additional Architectural Details

Several design choices merit explicit documentation:

Embedding normalization. An RMSNorm is applied immediately after the embedding lookup, before the first HybridBlock. This stabilizes the initial hidden state magnitude regardless of token frequency.

Partial RoPE. Only 25% of the head dimension receives rotary position embeddings (`partial_rotary_factor=0.25`), following the Qwen 3.5 pattern. The remaining 75% of dimensions are position-agnostic, allowing the model to learn position-independent features alongside positional ones.

GDN V expansion. The GDN layers use a value expansion ratio of 2 (`gdn_expand_v=2`), doubling the value head dimension relative to the key/query heads. This provides richer recurrent state without increasing the gate computation.

Residual lambda initialization. The learnable residual scaling factors (Section 3.4) are initialized to $\sqrt{1.1} \approx 1.049$, providing a slight residual amplification bias that empirically improves early training stability.

Tied embeddings. The input embedding matrix and the output projection (LM head) share weights, reducing the parameter count by one full vocabulary projection ($\text{dim} \times 151,936$). The shared matrix is optimized at the embedding learning rate.

3.6 Hardware Cache Exploitation

The Chimera topology creates a natural hardware advantage on modern GPUs. Because the top section reuses the same P physical blocks across L loops, the total weight footprint in fast memory during the top-section forward pass is just P blocks—not $P \times L$.

For the 510M model, the 4 physical top blocks total approximately 35 MB of bf16 parameters. The NVIDIA H100’s L2 cache is 50 MB. This means the entire shared section fits in L2 during the Ouroboros loops: the GPU fetches the top-block weights from HBM once on the first loop, and subsequent loops execute entirely from L2 cache with near-zero memory latency. This transforms a memory-bandwidth-bound operation into a compute-bound one, explaining why the 510M model achieves 50K+ tok/s on H100 despite its 787M virtual parameter equivalent.

The effect is scale-dependent: the 279M model’s top blocks (~18 MB) fit in L2 with room to spare, but the model is already compute-bound. The 1.1B model’s top blocks (~90 MB) exceed L2 capacity, forcing partial HBM fetches on each loop—visible in the throughput drop from 50K tok/s (510M on H100) to 32K tok/s (1.1B on B200).

3.7 The Vocabulary Tradeoff

We use the Qwen 3 tokenizer (151,936 tokens), dramatically oversized for small models. Tao et al. [2024] predict the optimal vocabulary for a 124M model is approximately 32K tokens. Our embedding matrix alone exceeds the reasoning core at the 279M scale.

This is intentional. The large vocabulary serves as a computational offload: by encoding 4–7 bytes per token, the model spends zero FLOPs learning to spell, parse morphology, or handle multilingual text. The reasoning core receives clean, high-level semantic tokens and can dedicate 100% of its capacity to logic and structure.

The cost is that the embedding matrix dominates the parameter budget at small scale and requires special optimization treatment (Section 5.1). As model width increases, the embedding fraction decreases and this tradeoff becomes more favorable.

4 Experimental Setup

4.1 Training Configuration

- **Optimizer:** Muon for 2D weight matrices, AdamW for embeddings and scalars
- **Learning rate:** 8×10^{-4} (Muon), 4×10^{-4} (embeddings), 8×10^{-5} (scalars)—see Section 5.1
- **Schedule:** WSD (Warmup-Stable-Decay), 100-step warmup, 10% cosine warmdown
- **Batch size:** 16 (32,768 tokens/step) on H100/B200; 32 on Spark
- **Precision:** bfloat16 with `torch.compile` (or Liger fused CE without compile)
- **Total budget:** 250,000 steps per model (~ 8.2 B tokens at batch 32, ~ 4.1 B at batch 16)

4.2 Data

Mixed pretraining across four sources, deterministically sampled via Knuth hash:

Table 2: Pretraining data mixture.

Source	Ratio	Tokens	Purpose
FineWeb-Edu	75%	3.75B	General knowledge, grammar
StarCoderData (Py/JSON/MD)	18%	900M	Sequential logic, bracket matching
FineMath-4+	5%	250M	Numerical reasoning
UltraChat 200k (ChatML)	2%	100M	Conversational structure, stop tokens

The UltraChat data is formatted using a patched Qwen 3 chat template that allows system messages at any position, producing native `<|im_start|>`, `<|im_end|>`, and `<think></think>` tokens during pretraining.

4.3 Hardware

Table 3: Hardware configurations and throughput.

GPU	Model	VRAM	Throughput	Cost/hr
DGX Spark (GB10)	279M	~ 15 GB	9K tok/s	— (owned)
NVIDIA H100 80GB	510M	64 GB	50K tok/s	\$2.20
NVIDIA B200 180GB	279M	50 GB	93K tok/s	\$5.00
NVIDIA B200 180GB	1.1B	107 GB	32K tok/s	\$5.00

The B200 achieves lower throughput than H100 on the 279M model due to compute underutilization—the small model cannot saturate the B200’s wider memory bus. At 1.1B (107 GB VRAM, 881W), the B200 reaches 99% utilization, indicating the architecture transitions from compute-bound to memory-bound between 510M and 1.1B on Blackwell hardware.

5 Results and Findings

5.1 The Embedding Learning Rate Bottleneck

The most significant optimization finding is a pathology specific to models with oversized vocabularies relative to their reasoning core.

With the initial configuration (Muon at 8×10^{-4} , all AdamW parameters at 8×10^{-5}), the validation loss exhibited premature flattening—training loss remained healthy but generalization stalled. The cause: the embedding matrix (155M parameters, 56% of the 279M model) ran at 8×10^{-5} while the Muon reasoning core ran at 8×10^{-4} —a $10\times$ disparity. The reasoning layers learned to process tokens faster than the embedding could learn to represent them.

Fix: Decoupled three-way optimizer with embedding-specific learning rate at 4×10^{-4} ($5\times$ increase). Validation loss dropped from 4.06 to 3.59 within 4,500 steps.

This finding likely generalizes to any architecture where the embedding matrix substantially outweighs the transformer layers—a common situation with modern large-vocabulary tokenizers (100K+) combined with small models.

5.2 Multi-Scale Training Results

All models use the same WSD schedule with warmdown at the final 10% of training. Results below are from the stable phase (constant LR), before warmdown.

Table 4: Validation loss progression across scales (selected checkpoints).

Steps	Tokens	279M	510M	1.1B
500	16M	—	4.01	5.08
5,000	164M	—	3.80	3.96
9,000	295M	—	3.67	3.56
17,000	557M	—	3.42	—
32,500	1.06B	3.86	—	—
42,000	1.38B	3.51	—	—
49,000	1.60B	—	—	3.29
52,000	1.70B	3.44	—	—
58,500	1.92B	3.41	—	—
70,500	2.31B	—	3.20	—
72,000	2.36B	—	—	3.27
103,500	3.39B	—	3.16	—

Key observations:

1. The 510M model leads all configurations at val loss **3.16** (step 103.5K, ~ 3.39 B tokens), having dropped continuously from 4.01 with consistent new-best checkpoints across 200+ evaluations.
2. The 1.1B model at 72K steps (2.36B tokens) has reached 3.27, closing in on the 510M despite fewer tokens seen. Per-token convergence is faster at larger width.
3. Width scaling confirmed: at comparable token counts (~ 1.9 B), the 279M sits at 3.41 while the 510M was already at ~ 3.24 and the 1.1B at ~ 3.27 . Per-token efficiency improves dramatically with model width.

5.3 WSD Schedule Behavior

All three models exhibit characteristic validation loss flattening during the WSD stable phase, with per-checkpoint improvements decreasing over training. This is expected: the constant high learning rate forces the optimizer to explore broadly rather than settle into local minima. The stable phase accumulates data entropy into the weights. The warmdown phase (steps 225K–250K) allows the optimizer to settle into precise basins, typically producing a steep loss drop.

None of the models have reached the warmdown phase yet. Final convergence results pending.

5.4 Emergent ChatML Behavior

With only 2% UltraChat dialogue data (100M tokens out of 5B total), all three models learn to generate native ChatML stop tokens (`<|im_end|>`) at contextually appropriate positions. The 510M model at 17.5K steps spontaneously terminates generation with the correct stop token in conversational text.

The models also produce empty `<think></think>` blocks in assistant-role completions—the same convention used by Nemotron-Cascade 2 [Yang et al., 2026] for non-thinking mode. This pretraining-level ChatML internalization should significantly reduce the SFT data budget needed for instruction-following behavior.

5.5 Qualitative Behavior at 510M (2.31B tokens)

At 2.31B tokens, the 510M model exhibits: structural fluency (correct markdown, numbered lists, academic citation format), valid Python syntax with class definitions and imports, multi-paragraph coherence over 300+ tokens, register switching across medical/academic/literary prompts, fabricated but properly formatted academic references, and full ChatML compliance when prompted with `<|im_start|>` format.

5.6 Sampling Parameter Evolution

The Ouroboros loop creates characteristic sampling challenges due to “attractor wells”—the shared weights create stronger token persistence than standard transformers, causing the hidden state to converge on locally dominant tokens.

Table 5: Sampling parameter sweep at 510M, step 70.5K.

Config	T	rep_pen	pres_pen	Loops?	Quality
No penalties	0.7	1.0	0.0	Yes	Loops on low-diversity prompts
Low temp	0.5	1.0	0.0	Yes	Severe looping
Low temp + pen	0.5	1.0	1.5	Yes	Peaked dist. overcomes penalties
Double Sledge	0.7	1.15	2.5	No	Forced topic jumps, over-diverse
Qwen Casual	0.7	1.0	1.5	No	Clean, focused, best overall
Qwen Creative	1.0	1.0	1.5	No	More creative, wider vocabulary

The attractor wells persist even at 2.3B tokens—they are *architectural*, not a training artifact. Presence penalty ≥ 1.5 is required to prevent looping regardless of training stage. The optimal configuration converges to Qwen 3’s default: $T=0.7$, $\text{top_k}=20$, $\text{top_p}=0.95$, $\text{rep_pen}=1.0$, $\text{presence_pen}=1.5$.

6 Limitations

1. **No controlled ablation against dense baseline.** We have not trained an equivalent dense transformer on identical data to isolate the contribution of weight sharing.
2. **Warmdown phase not yet reached.** All models are in the WSD stable phase. Final convergence results pending.
3. **No downstream evaluation.** We report pretraining metrics only. MMLU, HumanEval, and instruction-tuning benchmarks are planned.
4. **Vocabulary oversizing not ablated.** Whether the 152K vocabulary’s semantic offload benefit outweighs the embedding cost requires comparison with 32K vocabulary at matched compute.

5. **Single-GPU training only.** Multi-GPU scaling with the Chimera topology has not been validated.
6. **Context limited to 2048.** RoPE extension and context scaling are planned for post-pretraining.

7 Conclusion

The Chimera Topology demonstrates that intelligent architectural decomposition—splitting the network into parsing and reasoning zones, exploiting the recurrence tolerance of GDN layers, and using a large vocabulary as a computational offload—produces efficient models that scale well from 279M to 1.1B parameters. The multi-scale experiments confirm that wider models converge faster in token efficiency, with the 510M model reaching better loss in less than half the tokens of the 279M.

The embedding learning rate bottleneck finding is immediately actionable for any practitioner combining large vocabularies with small models. The emergent ChatML behavior from 2% dialogue data suggests that minimal conversational structure in pretraining can dramatically reduce downstream SFT requirements.

Training continues across three GPU configurations. We will update this document with warmdown-phase results, downstream evaluation scores, and context extension experiments.

8 Future Work: Chimera-MoE

8.1 Motivation

Standard Mixture-of-Experts architectures duplicate expert FFN blocks across every layer, causing total parameter counts to explode (e.g., Mixtral $8 \times 7\text{B}$ requires 47B total for 13B active). NVIDIA’s Nemotron-Cascade 2 [Yang et al., 2026] demonstrated that 3B active parameters suffice for gold-medal performance on IMO 2025 and IOI 2025, but their 30B MoE requires multi-GPU deployment.

The Chimera topology offers a natural solution: apply MoE routing only to the bottom (unique) layers, while keeping the top (shared) layers dense.

8.2 Architecture

The design separates specialization from reasoning:

Bottom layers (MoE, unique): Each bottom layer contains N expert FFN blocks with a learned top- K router. Only K experts activate per token. These layers act as domain-specialized feature extractors—parsing raw tokens from different domains into a unified semantic representation.

Top layers (dense, shared, looped): The Ouroboros shared blocks remain dense—no expert duplication. They receive the unified representation from the bottom and apply iterative reasoning through L loops. Because these blocks are physically small and shared, they continue to fit in L2 cache regardless of the number of bottom experts.

Table 6: Projected Chimera-MoE configuration (8 experts, $K=2$).

Section	Params	Active	Notes
Bottom (8 experts, $K=2$)	$\sim 4\text{B}$	$\sim 1\text{B}$	$8 \times 510\text{M}$ FFN experts
Top (shared, 3 loops)	$\sim 500\text{M}$	$\sim 500\text{M}$	Dense, fits L2 cache
Total	$\sim 4.5\text{B}$	$\sim 1.5\text{B}$	Virtual: $\sim 6\text{B}+$

MoE and Ouroboros address orthogonal efficiency axes: MoE reduces active compute via sparse activation; Ouroboros reduces total parameters via weight sharing. Combined, the model achieves extreme virtual-to-active ratios: Nemotron-Cascade requires 30B total for 3B active (multi-GPU); Chimera-MoE projects 4.5B total for 1.5B active with 6B+ virtual depth (single consumer GPU at 50+ tok/s).

8.3 Training Strategy

The preferred approach is **upcycling**: take a trained dense Auron checkpoint, duplicate each bottom-layer FFN N times to create identical experts, initialize the router randomly, and continue pretraining for ~ 500 B tokens with an auxiliary load-balancing loss ($\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{lm}} + \alpha \cdot \mathcal{L}_{\text{balance}}$, $\alpha \approx 0.01$). The top shared layers transfer directly—they are already trained as a universal reasoning engine. This preserves months of dense pretraining investment while adding specialization cheaply.

References

- J. Bae et al. Relaxed Recursive Transformers. *arXiv:2410.20672*, 2024.
- L. Ben Allal et al. SmolLM2. *arXiv:2502.02737*, 2025.
- J. Bernstein and D. Newhouse. Old Optimizer, New Norm. *arXiv:2409.20325*, 2024.
- S. De et al. Griffin. *arXiv:2402.19427*, 2024.
- M. Dehghani et al. Universal Transformers. *arXiv:1807.03819*, 2018.
- P. Glorioso et al. Zamba. *arXiv:2405.16712*, 2024.
- P. Glorioso et al. Zamba2 Suite. *arXiv:2411.15242*, 2024.
- J. Hoffmann et al. Chinchilla: Training Compute-Optimal Large Language Models. *arXiv:2203.15556*, 2022.
- P. Hsu et al. Liger-Kernel. *arXiv:2410.10989*, 2024.
- K. Jordan et al. Muon. <https://kellerjordan.github.io/posts/muon/>, 2024.
- Z. Lan et al. ALBERT: A Lite BERT. *arXiv:1909.11942*, 2019.
- O. Lieber et al. Jamba. *arXiv:2403.19887*, 2024.
- S. Liu et al. MobileLLM. *arXiv:2402.14905*, 2024.
- Z. Liu et al. Muon is Scalable. *arXiv:2502.16982*, 2025.
- N. Saunshi et al. Reasoning with Latent Thoughts. *arXiv:2502.17416*, 2025.
- C. Tao et al. Scaling Laws with Vocabulary. *arXiv:2407.13623*, 2024.
- E. Wijmans et al. Cut Cross-Entropy. *arXiv:2411.09009*, 2024.
- S. Yang et al. Gated Linear Attention. *arXiv:2312.06635*, 2023.
- S. Yang et al. Gated Delta Networks. *arXiv:2412.06464*, 2024.
- Z. Yang et al. Nemotron-Cascade 2: Post-Training LLMs with Cascade RL and Multi-Domain On-Policy Distillation. *arXiv:2603.19220*, 2026.
- Y. Zhu et al. Ouro. *arXiv:2510.25741*, 2025.